Practical 1

Part B	
1)	1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator. 2. Create a public class named MiniBank that contains the main method. 3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header. 4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer. 5. Display the numbered menu to the user and read the chosen number with a Scanner. 6. Use a switch expression on the choice to print a short placeholder for that option (for example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message.
	Create a private repository oop-minibank-25AIML057 and add the faculty collaborator using the faculty GitHub username. Install JDK 17 or newer. Save this stage in lab-01. Compile: javac MiniBank.java Run: java MiniBank Commit and tag: git add . then git commit -m "Practical 1" then git tag lab-01. README: Compile with javac MiniBank.java and run with java MiniBank. Select 1 to 5; option 5 exits.
	MiniBank.java
Code	

```
import java.util.Scanner;
record BankInfo(String name, String branch) {}
enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT }
public class MiniBank {
    public static void main(String[] args) {
        System.out.println(new BankInfo("MiniBank", "Anand"));
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer\n5. Exit");
            int choice;
            try { choice = Integer.parseInt(sc.nextLine()); }
            catch (NumberFormatException e) { System.out.println("Invalid choice"); continue; }
            if (choice < 1 || choice > 5) { System.out.println("Invalid choice"); continue; }
            MenuOption option = MenuOption.values()[choice - 1];
            String message = switch (option) {
                case OPEN_ACCOUNT -> "Open account - to be implemented later";
                case DEPOSIT -> "Deposit - to be implemented later";
                case WITHDRAW -> "Withdraw - to be implemented later";
                case TRANSFER -> "Transfer - to be implemented later";
                case EXIT -> "Goodbye";
            };
            System.out.println(message);
            if (option == MenuOption.EXIT) { break; }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>javac MiniBank.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>java MiniBank
BankInfo[name=MiniBank, branch=Anand]
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
1
Open account - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
2
Deposit - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
3
Withdraw - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
4
Transfer - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
5
Goodbye

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>
```

Practical 2

Part B	
1)	1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as “CUST101”. Add a constructor and public getter methods. 2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example “AC0001”). 3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance. 5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.
	Continue with Practical 1. Add these files; keep MiniBank.java. javac *.java java Lab02 Commit and tag lab-02.
	Customer.java

Code	public class Customer {
	private String name, email, mobile;
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	Account.java

Code	public class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	public Account(String ownerName) { this(ownerName, 0); }
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && amount <= balance) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }

```
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

Lab02.java

Code

```
public class Lab02 {
    public static void main(String[] args) {
        Customer customer = new Customer("Tej", "tej@test.com", "9876543210");
        System.out.println(customer.getCustomerId() + " " + customer.getName());
        Account[] accounts = {new Account("Riya",5000),new Account("Tej"),new
Account("Aman",2000)};
        accounts[0].deposit(2000); System.out.println(accounts[0].getBalance());
        System.out.println(accounts[0].withdraw(3000));
        System.out.println(accounts[0].getBalance());
        System.out.println(accounts[0].withdraw(10000));
        for (Account a : accounts) {
            System.out.println(a.getAccountNumber() + " " + a.getBalance());
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB02>javac *.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB02>java Lab02
CUST101 Tej
7000
true
4000
false
AC0001 4000
AC0002 0
AC0003 2000

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB02>
```

Practical 3

Part B	
1)	1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.
	Replace Account.java and Customer.java from Practical 2. Add Lab03.java. The Account copy constructor is for the equality demonstration; do not register copies as new accounts. Address is immutable, so sharing it in clone is safe. javac *.java java Lab03 Commit and tag lab-03.
	Account.java

Code	public class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	public Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && amount <= balance) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	public long getBalance() { return balance; }
	public boolean isActive() { return active; }
	}

	Customer.java
Code	public class Customer implements Cloneable { public static class Address {

```
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
}
```

Lab03.java

Code

```
public class Lab03 {
    public static void main(String[] args) {
        Account first = new Account("Riya",4000);
        Account copy = new Account(first);
        System.out.println(first); System.out.println(copy);
        System.out.println(first.equals(copy));
        Object obj = first; System.out.println(obj instanceof Account);
        Customer c = new Customer("Tej","tej@test.com","9876543210",
            new Customer.Address("Main Road","Anand","388001"));
        Customer clone = c.clone();
        System.out.println(clone.getName() + " " + clone.getAddress().getCity());
        System.out.println("Separate customer object: " + (c != clone));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>javac *.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>java Lab03
AC0001 | Riya | 4000 | true
AC0001 | Riya | 4000 | true
true
true
Tej Anand
Separate customer object: true

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>
```

Practical 4

Part B	
1)	1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean. 2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount. 3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as “DEPOSIT AC0001 500” with split(), converts the parts, and returns a Command object. 4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string. 5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.
	Add these files to the Practical 3 project. javac *.java java Lab04 Commit and tag lab-04.
	Validator.java

Code	import java.util.regex.Pattern;
	public class Validator {
	private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
	private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
	private static final Pattern PAN = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
	private static final Pattern IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
	private static boolean matches(Pattern p, String value) {
	return value != null && p.matcher(value).matches();
	}
	public static boolean isValidMobile(String s) { return matches(MOBILE,s); }
	public static boolean isValidEmail(String s) { return matches(EMAIL,s); }
	public static boolean isValidPan(String s) { return matches(PAN,s); }
	public static boolean isValidIfsc(String s) { return matches(IFSC,s); }
	}

	TransactionType.java
Code	public enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER }
	Command.java
Code	public record Command(TransactionType type, String accountNumber, long amount) {}
	CommandParser.java

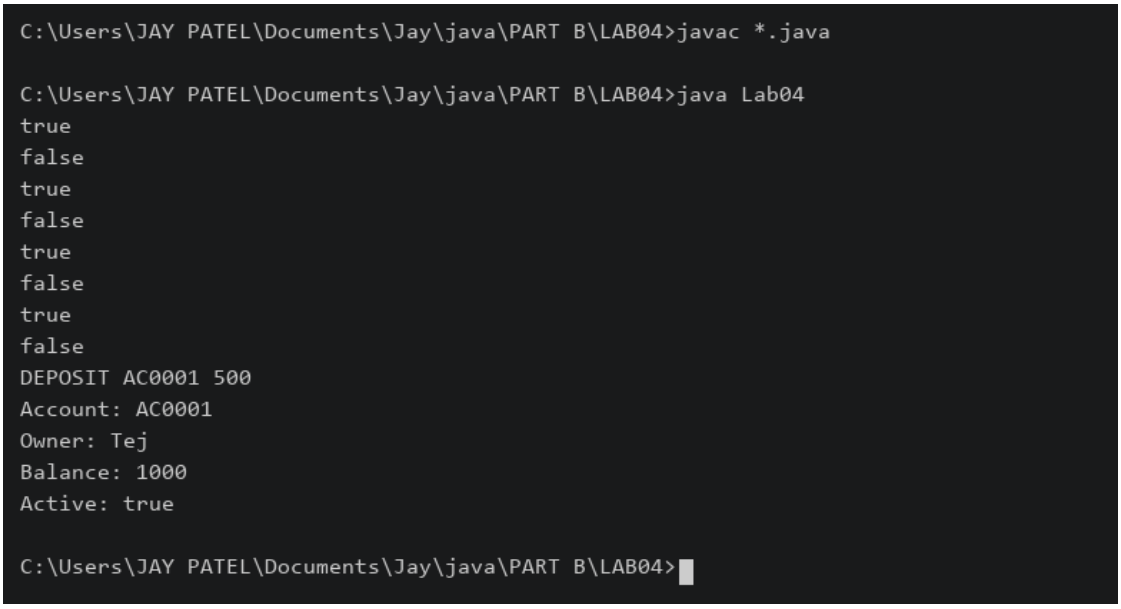
Code	public class CommandParser {
	public static Command parse(String line) {
	String[] parts = line.trim().split("\\s+");
	if (parts.length != 3) { throw new IllegalArgumentException("Use TYPE ACCOUNT AMOUNT"); }
	long amount = Long.parseLong(parts[2]);
	if (amount <= 0) { throw new IllegalArgumentException("Amount must be positive"); }
	return new Command(TransactionType.valueOf(parts[0].toUpperCase()),parts[1],amount);
	}
	}

	StatementFormatter.java

Code	public class StatementFormatter {
	  public static String buildStatement(Account a) {
	  return new StringBuilder().append("Account: ").append(a.getAccountNumber())
	  .append("\nOwner: ").append(a.getOwnerName()).append("\nBalance: ")
	  .append(a.getBalance()).append("\nActive: ").append(a.isActive()).toString();
	  }
	}

	Lab04.java

Code	public class Lab04 {
	  public static void main(String[] args) {
	  System.out.println(Validator.isValidMobile("9876543210"));
	  System.out.println(Validator.isValidMobile("12345"));
	  System.out.println(Validator.isValidEmail("abc@xyz.com"));
	  System.out.println(Validator.isValidEmail("abc"));
	  System.out.println(Validator.isValidPan("ABCDE1234F"));
	  System.out.println(Validator.isValidPan("1234"));
	  System.out.println(Validator.isValidIfsc("SBIN0001234"));
	  System.out.println(Validator.isValidIfsc("INVALID"));
	  Command c = CommandParser.parse("DEPOSIT AC0001 500");
	  System.out.println(c.type() + " " + c.accountNumber() + " " + c.amount());
	  System.out.println(StatementFormatter.buildStatement(new Account("Tej",1000)));
	  }
	}

Output	

Practical 5

Part B	
1)	1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount). 2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance. 3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit. 4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked). 5. Call super(...) from each subclass constructor to initialise the inherited fields. 6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially.
	Replace Account.java and add the subclasses and Lab05.java. Keep earlier driver files in their earlier lab folders; they instantiate the now abstract Account. Compile only the current model, utility files and driver. javac Account.java Customer.java SavingsAccount.java CurrentAccount.java FixedDepositAccount.java Validator.java TransactionType.java Command.java CommandParser.java StatementFormatter.java Lab05.java java Lab05 Commit and tag lab-05.
	Account.java

Code	public abstract class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
	return false;
	}

	public String getAccountNumber() { return accountNumber; } public String getOwnerName() { return ownerName; } public long getBalance() { return balance; } public boolean isActive() { return active; } }
	SavingsAccount.java

Code	public class SavingsAccount extends Account {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	public double interestRate() { return 4.0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	CurrentAccount.java

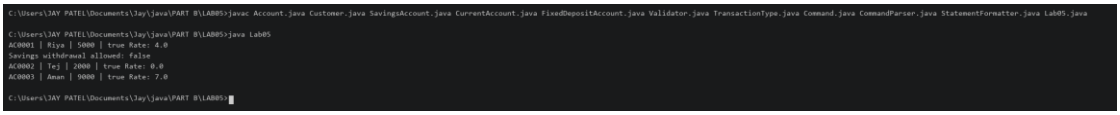
Code	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	public double interestRate() { return 0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	FixedDepositAccount.java

Code	public class FixedDepositAccount extends Account {
	public FixedDepositAccount(String name, long balance) { super(name,balance); }
	public double interestRate() { return 7.0; }
	public boolean canWithdraw(long amount) { return false; }
	}

	Lab05.java

Code	public class Lab05 {
	public static void main(String[] args) {
	Account[] accounts = {new SavingsAccount("Riya",5000,1000),
	new CurrentAccount("Tej",2000,1000),new FixedDepositAccount("Aman",9000)};
	for (Account a : accounts) {
	System.out.println(a + " Rate: " + a.interestRate());
	if (a instanceof SavingsAccount s) {
	System.out.println("Savings withdrawal allowed: " + s.canWithdraw(4500));
	}
	}
	}
	}

Output	

Practical 6

Part B	
1)	1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it. 2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance. 3. Create a functional interface named WithdrawRule with a single method boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression. 4. Create a marker interface named Premium (an interface with no methods). 5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import. 6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar.
	Move the current project into these packages; use the complete files below instead of the earlier default-package files. Keep old lab drivers outside this source tree. javac -d out model/*.java util/*.java service/*.java java -cp out service.Lab06 jar cfm minibank.jar manifest.mf -C out . java -jar minibank.jar Keep the blank line at the end of manifest.mf. Commit and tag lab-06.
	model/Account.java

Code	package model;
	public abstract class Account implements Transactable, InterestBearing {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	public long getBalance() { return balance; }

	public boolean isActive() { return active; } }
	model/Customer.java

Code	package model;
	public class Customer implements Cloneable {
	public static class Address {
	private final String line, city, pincode;
	public Address(String line, String city, String pincode) {
	this.line = line; this.city = city; this.pincode = pincode;
	}
	public String getLine() { return line; }
	public String getCity() { return city; }
	public String getPincode() { return pincode; }
	}
	private Address address;
	public Customer(String name, String email, String mobile, Address address) {
	this(name,email,mobile); this.address = address;
	}
	public Address getAddress() { return address; }
	public Customer clone() {
	try { return (Customer) super.clone(); }
	catch (CloneNotSupportedException e) { throw new AssertionError(e); }
	}
	private String name, email, mobile;
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	public double interestRate() { return 4.0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	public double interestRate() { return 0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java
Code	package model; public class FixedDepositAccount extends Account {

	public FixedDepositAccount(String name, long balance) { super(name,balance); } public double interestRate() { return 7.0; } public boolean canWithdraw(long amount) { return false; } }
	model/TransactionType.java
Code	package model; public enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER }
	model/Command.java
Code	package model; public record Command(TransactionType type, String accountNumber, long amount) {}
	model/Transactable.java

Code	package model;
	public interface Transactable {
	void deposit(long amount);
	boolean withdraw(long amount);
	}

	model/InterestBearing.java

Code	package model;
	public interface InterestBearing {
	double interestRate();
	long getBalance();
	default double yearlyInterest() { return getBalance() * interestRate() / 100; }
	}

	model/Premium.java
Code	package model; public interface Premium {}
	util/Validator.java

Code	package util;
	import java.util.regex.Pattern;
	public class Validator {
	private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
	private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
	private static final Pattern PAN = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
	private static final Pattern IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
	private static boolean matches(Pattern p, String value) {
	return value != null && p.matcher(value).matches();
	}
	public static boolean isValidMobile(String s) { return matches(MOBILE,s); }
	public static boolean isValidEmail(String s) { return matches(EMAIL,s); }
	public static boolean isValidPan(String s) { return matches(PAN,s); }
	public static boolean isValidIfsc(String s) { return matches(IFSC,s); }
	}

	util/CommandParser.java

Code	package util;
	import model.*;
	
	public class CommandParser {
	public static Command parse(String line) {
	String[] parts = line.trim().split("\\s+");
	if (parts.length != 3) { throw new IllegalArgumentException("Use TYPE ACCOUNT AMOUNT"); }
	long amount = Long.parseLong(parts[2]);
	if (amount <= 0) { throw new IllegalArgumentException("Amount must be positive"); }
	return new Command(TransactionType.valueOf(parts[0].toUpperCase()),parts[1],amount);
	}
	}

	util/StatementFormatter.java

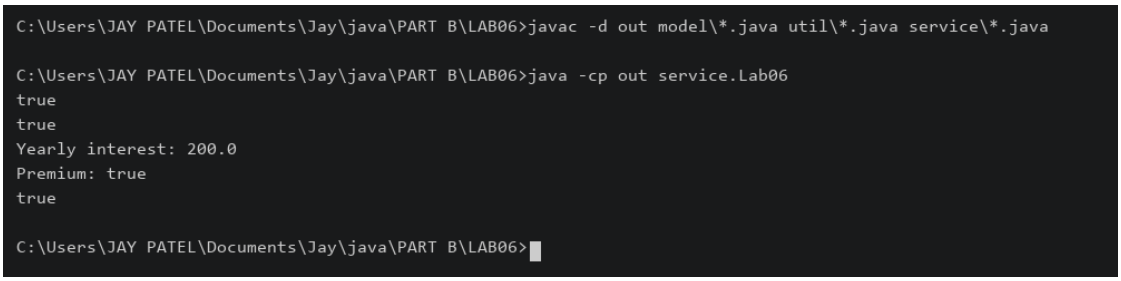
Code	package util;
	import model.Account;
	
	public class StatementFormatter {
	  public static String buildStatement(Account a) {
	  return new StringBuilder().append("Account: ").append(a.getAccountNumber())
	  .append("\nOwner: ").append(a.getOwnerName()).append("\nBalance: ")
	  .append(a.getBalance()).append("\nActive: ").append(a.isActive()).toString();
	  }
	}

	service/WithdrawRule.java
Code	package service; import model.Account; public interface WithdrawRule { boolean allow(Account account, long amount); }
	service/Lab06.java

Code	package service;
	import model.*;
	import static util.Validator.isValidMobile;
	public class Lab06 {
	  public static void main(String[] args) {
	  Account a = new SavingsAccount("Tej",5000,1000);
	  WithdrawRule first = new WithdrawRule() {
	  public boolean allow(Account a, long amount) { return a.canWithdraw(amount); }
	  };
	  WithdrawRule second = (account, amount) -> account.canWithdraw(amount);
	  System.out.println(first.allow(a,500));
	  System.out.println(second.allow(a,500));
	  System.out.println("Yearly interest: " + a.yearlyInterest());
	  System.out.println("Premium: " + (a instanceof Premium));
	  System.out.println(isValidMobile("9876543210"));
	  }
	}

	service/MiniBank.java

Code	package service;
	import java.util.Scanner;
	record BankInfo(String name, String branch) {}
	enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT }
	public class MiniBank {
	  public static void main(String[] args) {
	  System.out.println(new BankInfo("MiniBank", "Anand"));
	  Scanner sc = new Scanner(System.in);
	  while (true) {
	  System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer\n5. Exit");
	  int choice;
	  try { choice = Integer.parseInt(sc.nextLine()); }
	  catch (NumberFormatException e) { System.out.println("Invalid choice"); continue; }
	  if (choice < 1 || choice > 5) { System.out.println("Invalid choice"); continue; }
	  MenuOption option = MenuOption.values()[choice - 1];
	  String message = switch (option) {
	  case OPEN_ACCOUNT -> "Open account - to be implemented later";
	  case DEPOSIT -> "Deposit - to be implemented later";
	  case WITHDRAW -> "Withdraw - to be implemented later";
	  case TRANSFER -> "Transfer - to be implemented later";
	  case EXIT -> "Goodbye";
	  };
	  System.out.println(message);
	  if (option == MenuOption.EXIT) { break; }
	  }
	  }
	}

	manifest.mf
Text	Manifest-Version: 1.0 Main-Class: service.MiniBank
Output	

Practical 7

Part B	
1)	1. Add the @Override annotation above every overridden method written so far (toString, equals, hashCode and the subclass methods). Add @FunctionalInterface above the WithdrawRule interface. 2. In a package named model.annotation, define three custom annotations: @Id (a marker, with @Target(FIELD) and @Retention(RUNTIME)); @Positive with a member String message() default “must be > 0”; and @MaxLength with a member int value(). 3. Put @Id on the accountNumber field and @Positive on the balance field of Account. 4. Create a class named AnnotationValidator with a public static method String[] validate(Object obj) that uses reflection — getClass().getDeclaredFields(), field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the @Positive and @MaxLength rules and return the error messages as a String[]. 5. In main, create an Account with a negative balance and run AnnotationValidator.validate() to print the error.
	Continue from Practical 6. Replace matching files and add new files. The validator walks parent classes because balance is declared in Account, not SavingsAccount. javac -d out model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab07 Commit and tag lab-07.
	model/annotation/Id.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface Id {}
	model/annotation/Positive.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface Positive { String message() default "must be > 0"; }
	model/annotation/MaxLength.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface MaxLength { int value(); }
	model/Account.java

Code	package model;
	import model.annotation.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;

```
    }
    protected Account(Account other) {
        accountNumber = other.accountNumber;
        ownerName = other.ownerName; balance = other.balance; active = other.active;
    }
    @Override
    public String toString() {
        return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
    }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Account a)) { return false; }
        return accountNumber.equals(a.accountNumber);
    }
    @Override
    public int hashCode() { return accountNumber.hashCode(); }
    public Account(String ownerName) { this(ownerName, 0); }
    public abstract double interestRate();
    public abstract boolean canWithdraw(long amount);
    @Override
    public void deposit(long amount) {
        if (amount > 0) { balance += amount; }
    }
    @Override
    public boolean withdraw(long amount) {
        if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
        return false;
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

model/Customer.java

Code

```
package model;
public class Customer implements Cloneable {
    public static class Address {
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    @Override
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
```

	}
	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	public double interestRate() { return 0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java

Code	package model;
	public class FixedDepositAccount extends Account {
	public FixedDepositAccount(String name, long balance) { super(name,balance); }
	@Override
	public double interestRate() { return 7.0; }
	@Override
	public boolean canWithdraw(long amount) { return false; }
	}

	service/WithdrawRule.java
Code	package service; import model.Account; @FunctionalInterface public interface WithdrawRule { boolean allow(Account account, long amount); }
	util/AnnotationValidator.java

Code	package util;
	import model.annotation.*;
	import java.lang.reflect.Field;
	import java.util.ArrayList;
	public class AnnotationValidator {
	public static String[] validate(Object obj) {
	ArrayList<String> errors = new ArrayList<>();
	for (Class<?> type = obj.getClass(); type != null; type = type.getSuperclass()) {
	for (Field field : type.getDeclaredFields()) {
	if (!field.isAnnotationPresent(Positive.class)
	&& !field.isAnnotationPresent(MaxLength.class)) { continue; }
	try {
	field.setAccessible(true);
	Object value = field.get(obj);
	Positive p = field.getAnnotation(Positive.class);
	if (p != null && value instanceof Number n && n.doubleValue() <= 0) {
	errors.add(field.getName() + ": " + p.message());

```
                }
                MaxLength m = field.getAnnotation(MaxLength.class);
                if (m != null && value instanceof String s && s.length() > m.value()) {
                    errors.add(field.getName() + ": maximum length " + m.value());
                }
            } catch (IllegalAccessException e) { throw new IllegalStateException(e); }
        }
    }
    return errors.toArray(new String[0]);
  }
}
```

service/Lab07.java

Code

```
package service;
import model.*;
import util.AnnotationValidator;
public class Lab07 {
   public static void main(String[] args) {
      Account bad = new SavingsAccount("Tej",-100,0);
      for (String error : AnnotationValidator.validate(bad)) { System.out.println(error); }
      Account good = new SavingsAccount("Riya",1000,0);
      System.out.println("Valid errors: " + AnnotationValidator.validate(good).length);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>javac -d out model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>java -cp out service.Lab07
balance: must be > 0
Valid errors: 0

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>
```

Practical 8

Part B	
1)	1. In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall. 2. Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount. 3. Add a method transfer(Account to, long amount) that uses try, catch and finally and re-throws a BankException when needed. 4. In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create.
	Continue from Practical 7. Replace Account and Transactable so their checked exception declarations agree. The boolean return remains for successful withdrawals; failures now throw exceptions. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab08 Commit and tag lab-08.
	exception/BankException.java

Code	package exception;
	public class BankException extends Exception {
	public BankException(String message) { super(message); }
	}

	exception/InsufficientFundsException.java

Code	package exception;
	public class InsufficientFundsException extends BankException {
	private long shortfall;
	public InsufficientFundsException(long shortfall) {
	super("Withdrawal failed: short by " + shortfall); this.shortfall = shortfall;
	}
	public long getShortfall() { return shortfall; }
	}

	exception/InvalidAmountException.java

Code	package exception;
	public class InvalidAmountException extends BankException {
	public InvalidAmountException() { super("Amount must be positive"); }
	}

	exception/AccountNotFoundException.java

Code	package exception;
	public class AccountNotFoundException extends BankException {
	public AccountNotFoundException(String id) { super("Account not found: " + id); }
	}

	model/Transactable.java

Code	package model;
	import exception.*;
	public interface Transactable {
	void deposit(long amount) throws InvalidAmountException;
	boolean withdraw(long amount) throws InsufficientFundsException, InvalidAmountException;

	}
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;
	}
	@Override
	public boolean withdraw(long amount)
	throws InsufficientFundsException, InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	if (!canWithdraw(amount)) {
	long available = Math.max(0, withdrawalCapacity());
	throw new InsufficientFundsException(Math.max(0,amount - available));
	}
	balance -= amount; return true;
	}
	public void transfer(Account to, long amount) throws BankException {
	try {
	if (to == null) { throw new AccountNotFoundException("destination"); }
	if (to == this) { throw new BankException("Choose a different account"); }
	withdraw(amount); to.deposit(amount);
	} catch (BankException e) {
	System.out.println("Transfer failed: " + e.getMessage()); throw e;
	} finally { System.out.println("Transfer attempt completed"); }
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	@Override
	public long getBalance() { return balance; }
	public boolean isActive() { return active; }
	}

	service/Lab08.java

Code	package service;
	import model.*;
	import exception.*;
	class BankSession implements AutoCloseable {
	public void close() { System.out.println("Session closed"); }
	}
	public class Lab08 {
	public static void main(String[] args) {
	Account a = new SavingsAccount("Tej",1000,0);
	Account b = new SavingsAccount("Riya",500,0);
	try (BankSession session = new BankSession()) {
	try { a.withdraw(5000); }
	catch (InsufficientFundsException e) { System.out.println(e.getMessage()); }
	try { a.deposit(0); }
	catch (InvalidAmountException e) { System.out.println(e.getMessage()); }
	try { a.transfer(null,100); }
	catch (BankException e) { System.out.println(e.getMessage()); }
	a.transfer(b,200);
	System.out.println(a); System.out.println(b);
	} catch (BankException e) { System.out.println(e.getMessage()); }
	finally { System.out.println("Bank operations completed"); }
	}
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() - minBalance); }
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() + overdraftLimit); }
	@Override
	public double interestRate() { return 0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java
Code	package model; public class FixedDepositAccount extends Account { public FixedDepositAccount(String name, long balance) { super(name,balance); } @Override protected long withdrawalCapacity() { return 0; } @Override public double interestRate() { return 7.0; }

```
    @Override
    public boolean canWithdraw(long amount) { return false; }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>java -cp out service.Lab08
Withdrawal failed: short by 4000
Amount must be positive
Transfer failed: Account not found: destination
Transfer attempt completed
Account not found: destination
Transfer attempt completed
AC0001 | Tej | 800 | true
AC0002 | Riya | 700 | true
Session closed
Bank operations completed

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>
```

Practical 9

Part B	
1)	1. Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount). 2. In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition. 3. Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct. 4. Print the thread names to observe the thread lifecycle.
	First add AccountWorker and Lab09 while keeping the unsynchronized Account from Practical 8. Compile and run to observe a possible wrong balance; race results vary and may occasionally be correct. Then replace Account.java with the synchronized version below and repeat; the final balance must be 10000. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab09 Commit and tag lab-09.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;
	}
	@Override

```
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            withdraw(amount); to.deposit(amount);
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage()); throw e;
        } finally { System.out.println("Transfer attempt completed"); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/AccountWorker.java

Code

```
package service;
import model.Account;
import exception.InvalidAmountException;
public class AccountWorker implements Runnable {
    private Account account;
    private int times;
    private long amount;
    public AccountWorker(Account account, int times, long amount) {
        this.account = account; this.times = times; this.amount = amount;
    }
    public void run() {
        System.out.println(Thread.currentThread().getName() + " started");
        try { for (int i = 0; i < times; i++) { account.deposit(amount); } }
        catch (InvalidAmountException e) { System.out.println(e.getMessage()); }
        System.out.println(Thread.currentThread().getName() + " finished");
    }
}
```

service/Lab09.java

Code

```
package service;
import model.*;
public class Lab09 {
    public static void main(String[] args) throws Exception {
        Account a = new SavingsAccount("Tej",0,0);
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(new AccountWorker(a,1000,1),"Worker-" + i);
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        System.out.println("Final balance: " + a.getBalance());
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>java -cp out service.Lab09
Worker-4 started
Worker-7 started
Worker-6 started
Worker-2 started
Worker-6 finished
Worker-2 finished
Worker-3 started
Worker-5 started
Worker-3 finished
Worker-5 finished
Worker-4 finished
Worker-8 started
Worker-7 finished
Worker-8 finished
Worker-9 started
Worker-1 started
Worker-9 finished
Worker-0 started
Worker-1 finished
Worker-0 finished
Final balance: 10000

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>
```

Practical 10

Part B	
1)	1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination(). 2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion. 3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks. 4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first.
	Continue from Practical 9. Run service.DeadlockDemo separately to demonstrate the deliberate hang; stop with Ctrl+C. Run service.Lab10 for the fixed account-number lock order, thread pool and buffer. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.DeadlockDemo java -cp out service.Lab10 Commit and tag lab-10.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;

```
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/TransactionProcessor.java

Code

```
package service;
import java.util.concurrent.*;
public class TransactionProcessor {
    private ExecutorService pool = Executors.newFixedThreadPool(4);
    public void submit(Runnable task) { pool.execute(task); }
    public void stop() throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
            pool.shutdownNow();
            if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
                throw new IllegalStateException("Tasks did not stop");
            }
        }
    }
}
```

service/TransactionBuffer.java

Code

```
package service;
public class TransactionBuffer {
    private Runnable[] buffer = new Runnable[3];
    private int head, tail, size;
    public synchronized void put(Runnable task) throws InterruptedException {
        while (size == buffer.length) { wait(); }
        buffer[tail] = task; tail = (tail + 1) % buffer.length; size++; notify();
    }
    public synchronized Runnable take() throws InterruptedException {
        while (size == 0) { wait(); }
        Runnable task = buffer[head]; buffer[head] = null;
        head = (head + 1) % buffer.length; size--; notify(); return task;
    }
}
```

	service/Lab10.java

Code	package service;
	import model.*;
	import exception.*;
	public class Lab10 {
	public static void main(String[] args) throws Exception {
	Account a = new SavingsAccount("Tej",10000,0);
	Account b = new SavingsAccount("Riya",10000,0);
	TransactionProcessor processor = new TransactionProcessor();
	for (int i = 0; i < 10; i++) {
	processor.submit(() -> {
	try { a.deposit(100); a.withdraw(50); }
	catch (BankException e) { System.out.println(e.getMessage()); }
	});
	}
	processor.stop();
	TransactionBuffer buffer = new TransactionBuffer();
	Thread producer = new Thread(() -> {
	try {
	for (int i = 0; i < 5; i++) {
	buffer.put(() -> {
	try { a.deposit(10); }
	catch (BankException e) { System.out.println(e.getMessage()); }
	});
	}
	} catch (InterruptedException e) { Thread.currentThread().interrupt(); }
	});
	Thread consumer = new Thread(() -> {
	try { for (int i = 0; i < 5; i++) { buffer.take().run(); } }
	catch (InterruptedException e) { Thread.currentThread().interrupt(); }
	});
	producer.start(); consumer.start(); producer.join(); consumer.join();
	Thread t1 = new Thread(() -> move(a,b));
	Thread t2 = new Thread(() -> move(b,a));
	t1.start(); t2.start(); t1.join(); t2.join();
	System.out.println(a); System.out.println(b);
	}
	static void move(Account from, Account to) {
	try { from.transfer(to,100); }
	catch (BankException e) { System.out.println(e.getMessage()); }
	}
	}

	service/DeadlockDemo.java

Code	package service;
	import model.*;
	import java.util.concurrent.CountDownLatch;
	public class DeadlockDemo {
	public static void main(String[] args) {
	Account a = new SavingsAccount("Tej",1000,0);
	Account b = new SavingsAccount("Riya",1000,0);
	CountDownLatch held = new CountDownLatch(2);
	new Thread(() -> badTransfer(a,b,held)).start();
	new Thread(() -> badTransfer(b,a,held)).start();
	}
	static void badTransfer(Account from, Account to, CountDownLatch held) {
	synchronized (from) {
	held.countDown();
	try { held.await(); }
	catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
	synchronized (to) {
	try { from.withdraw(100); to.deposit(100); }
	catch (Exception e) { System.out.println(e.getMessage()); }
	}
	}
	}
	}

Output	

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>javac -d out exception*.java model\annotation*.java model*.java util*.java service*.java
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>java -cp out service.Lab10
Transfer attempt completed
Transfer attempt completed
AC0001 | Tej | 10550 | true
AC0002 | Riya | 10000 | true
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>

Practical 11

Part B	
1)	1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field. 2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream. 3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND. 4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file. 5. In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report.
	Continue from Practical 10. Subclasses inherit Serializable. readObject restores the account-number counter after loading. Logs append, so running the demo again adds another set of log entries. Add data/ and out/ to .gitignore. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab11 Commit source only and tag lab-11.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing, java.io.Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);

```
    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        balance += amount;
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    private void readObject(java.io.ObjectInputStream in)
        throws java.io.IOException, ClassNotFoundException {
        in.defaultReadObject();
        long number = Long.parseLong(accountNumber.substring(2));
        synchronized (Account.class) { counter = Math.max(counter,number); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

model/Customer.java

Code

```
package model;
public class Customer implements Cloneable, java.io.Serializable {
    private static final long serialVersionUID = 1L;
    public static class Address implements java.io.Serializable {
        private static final long serialVersionUID = 1L;
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    @Override
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
```

```
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
}
```

util/StatePersister.java

Code

```
package util;
import model.Account;
import java.io.*;
import java.nio.file.*;
public class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
        Path parent = file.toAbsolutePath().getParent();
        Files.createDirectories(parent);
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(accounts);
        }
    }
    public static Account[] load(Path file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) in.readObject();
        }
    }
}
```

util/TransactionLog.java

Code

```
package util;
import java.nio.file.*;
import java.io.IOException;
public class TransactionLog {
    public static synchronized void append(Path file, String line) throws IOException {
        Files.createDirectories(file.toAbsolutePath().getParent());
        Files.writeString(file,line + System.lineSeparator(),
            StandardOpenOption.CREATE,StandardOpenOption.APPEND);
    }
}
```

util/ReportGenerator.java

Code

```
package util;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.io.*;
public class ReportGenerator {
    public static String generate(Path folder, Path reportFile) throws IOException {
        Files.createDirectories(folder);
        long deposits = 0, withdrawals = 0;
        StringBuilder details = new StringBuilder();
        try (DirectoryStream<Path> files = Files.newDirectoryStream(folder,"*.log")) {
            for (Path file : files) {
                if (!Files.isRegularFile(file) || Files.size(file) == 0) { continue; }
                BasicFileAttributes attr = Files.readAttributes(file,BasicFileAttributes.class);
                details.append(file.getFileName()).append(" Size: ").append(attr.size())
                    .append(" Modified: ").append(attr.lastModifiedTime()).append("\n");
                try (BufferedReader reader = Files.newBufferedReader(file)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        String[] p = line.trim().split("\\s+");
                        if (p.length != 3) { continue; }
                        try {
                            long amount = Long.parseLong(p[2]);
                            if (amount <= 0) { continue; }
```

	if (p[0].equals("DEPOSIT")) { deposits += amount; } else if (p[0].equals("WITHDRAW")) { withdrawals += amount; } } catch (NumberFormatException e) { continue; } } } } } details.append("Deposits: ").append(deposits).append("\nWithdrawals: ") .append(withdrawals).append("\nNet change: ").append(deposits - withdrawals); Files.createDirectories(reportFile.toAbsolutePath().getParent()); Files.writeString(reportFile,details); return details.toString(); } }
	service/Lab11.java

Code	package service;
	import model.*;
	import util.*;
	import java.nio.file.*;
	public class Lab11 {
	public static void main(String[] args) throws Exception {
	Account[] accounts = {new SavingsAccount("Tej",5000,0),
	new CurrentAccount("Riya",2000,1000),new FixedDepositAccount("Aman",9000)};
	accounts[0].deposit(500); accounts[1].withdraw(200);
	Path data = Path.of("data/accounts.dat");
	StatePersister.save(accounts,data);
	Account[] loaded = StatePersister.load(data);
	for (Account a : loaded) { System.out.println(a); }
	Path logs = Path.of("data/logs");
	TransactionLog.append(logs.resolve("daily.log"),"DEPOSIT "
	+ accounts[0].getAccountNumber() + " 500");
	TransactionLog.append(logs.resolve("daily.log"),"WITHDRAW "
	+ accounts[1].getAccountNumber() + " 200");
	System.out.println(ReportGenerator.generate(logs,Path.of("data/report.txt")));
	}
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private static final long serialVersionUID = 1L;
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() - minBalance); }
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private static final long serialVersionUID = 1L;
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() + overdraftLimit); }
	@Override
	public double interestRate() { return 0; }
	@Override

```
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }
}
```

model/FixedDepositAccount.java

Code

```
package model;
public class FixedDepositAccount extends Account {
    private static final long serialVersionUID = 1L;
    public FixedDepositAccount(String name, long balance) { super(name,balance); }
    @Override
    protected long withdrawalCapacity() { return 0; }
    @Override
    public double interestRate() { return 7.0; }
    @Override
    public boolean canWithdraw(long amount) { return false; }
}
```

Output



Practical 12

Part B	
1)	1. Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll(). 2. Keep a transaction history in an ArrayList<String>. 3. Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator. 4. Make Account implement Comparable<Account> by comparing account numbers in compareTo(). 5. In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe. 6. In main, add several accounts to the store and print them sorted by balance.
	Continue from Practical 11. AccountStore uses HashMap and ArrayList. TransactionProcessor uses ConcurrentHashMap while Account methods still synchronize balance updates. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab12 Commit and tag lab-12.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing, java.io.Serializable, Comparable<Account> {
	private static final long serialVersionUID = 1L;
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }

```
        balance += amount;
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    private void readObject(java.io.ObjectInputStream in)
        throws java.io.IOException, ClassNotFoundException {
        in.defaultReadObject();
        long number = Long.parseLong(accountNumber.substring(2));
        synchronized (Account.class) { counter = Math.max(counter,number); }
    }
    @Override
    public int compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/AccountStore.java

Code

```
package service;
import model.Account;
import java.util.*;
public class AccountStore {
    private HashMap<String,Account> accounts = new HashMap<>();
    private ArrayList<String> history = new ArrayList<>();
    public void add(Account account) { accounts.put(account.getAccountNumber(),account); }
    public Account get(String number) { return accounts.get(number); }
    public ArrayList<Account> getAll() { return new ArrayList<>(accounts.values()); }
    public void record(String line) { history.add(line); }
    public ArrayList<String> getHistory() { return new ArrayList<>(history); }
    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> list = getAll();
        list.sort(Comparator.comparingLong(Account::getBalance).reversed()); return list;
    }
}
```

service/TransactionProcessor.java

Code

```
package service;
import java.util.concurrent.*;
import model.Account;
import java.util.Collection;
public class TransactionProcessor {
```

```
    private ConcurrentHashMap<String,Account> accounts = new ConcurrentHashMap<>();
    public void add(Account a) { accounts.put(a.getAccountNumber(),a); }
    public Account get(String number) { return accounts.get(number); }
    public void useAccounts(Collection<Account> source) {
        accounts.clear();
        for (Account a : source) { add(a); }
    }
    private ExecutorService pool = Executors.newFixedThreadPool(4);
    public void submit(Runnable task) { pool.execute(task); }
    public void stop() throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
            pool.shutdownNow();
            if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
                throw new IllegalStateException("Tasks did not stop");
            }
        }
    }
}
```

service/Lab12.java

Code

```
package service;
import model.*;
public class Lab12 {
    public static void main(String[] args) throws Exception {
        AccountStore store = new AccountStore();
        Account a = new SavingsAccount("Tej",5000,0);
        Account b = new SavingsAccount("Riya",2000,0);
        Account c = new SavingsAccount("Aman",9000,0);
        store.add(a); store.add(b); store.add(c);
        store.record("Opened " + a.getAccountNumber());
        System.out.println(store.get(b.getAccountNumber()));
        for (Account item : store.sortedByBalance()) { System.out.println(item); }
        System.out.println(store.getHistory());
        TransactionProcessor processor = new TransactionProcessor();
        processor.useAccounts(store.getAll());
        for (int i = 0; i < 10; i++) {
            processor.submit(new AccountWorker(processor.get(a.getAccountNumber()),100,1));
        }
        processor.stop(); System.out.println(a);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>java -cp out service.Lab12
AC0002 | Riya | 2000 | true
AC0003 | Aman | 9000 | true
AC0001 | Tej | 5000 | true
AC0002 | Riya | 2000 | true
[Opened AC0001]
pool-1-thread-2 started
pool-1-thread-4 started
pool-1-thread-2 finished
pool-1-thread-1 started
pool-1-thread-2 started
pool-1-thread-2 finished
pool-1-thread-3 started
pool-1-thread-4 finished
pool-1-thread-1 finished
pool-1-thread-4 started
pool-1-thread-1 started
pool-1-thread-4 finished
pool-1-thread-1 finished
pool-1-thread-2 started
pool-1-thread-2 finished
pool-1-thread-3 finished
pool-1-thread-4 started
pool-1-thread-1 started
pool-1-thread-1 finished
pool-1-thread-4 finished
AC0001 | Tej | 6000 | true

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>
```

Practical 13

Part B	
1)	1. Create a generic class named Repository<T> that holds a HashMap<String, T>. Add methods T save(T entity), T findById(String id), ArrayList<T> findAll() and boolean delete(String id). 2. Inside save(), find the id by reflection: look for the field marked @Id (from Practical 7) and read its value as the map key. 3. Add a method marked @SafeVarargs called saveAll(T... items) that saves several objects at once. 4. Add one generic method with a bounded type parameter, for example static <N extends Number> double average(N[] values). 5. In main, create a Repository<Account>, save several accounts, fetch one by id, and print all of them.
	Continue from Practical 12. Repository scans inherited fields so the @Id on Account works for every subtype. Customer also gets @Id for reuse. Synchronized repository methods protect its HashMap during later batch access. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab13 Commit and tag lab-13.
	service/Repository.java

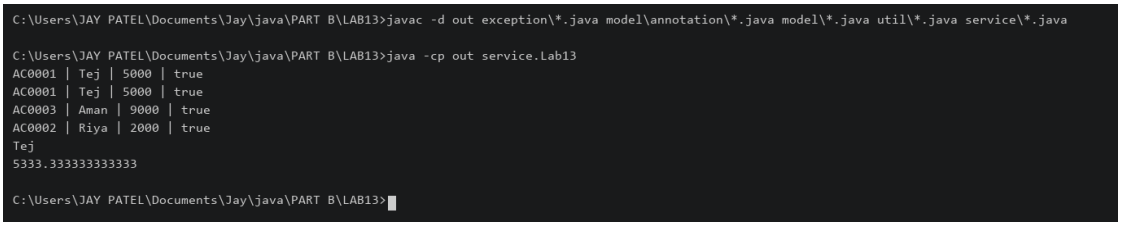
Code	package service;
	import model.annotation.Id;
	import java.lang.reflect.Field;
	import java.util.*;
	public class Repository<T> {
	private HashMap<String,T> data = new HashMap<>();
	private String idOf(T entity) {
	for (Class<?> type = entity.getClass(); type != null; type = type.getSuperclass()) {
	for (Field field : type.getDeclaredFields()) {
	if (field.isAnnotationPresent(Id.class)) {
	try {
	field.setAccessible(true);
	Object value = field.get(entity);
	if (value == null) { throw new IllegalArgumentException("Null id"); }
	return value.toString();
	} catch (IllegalAccessException e) { throw new IllegalStateException(e); }
	}
	}
	}
	throw new IllegalArgumentException("No @Id field");
	}
	public synchronized T save(T entity) { data.put(idOf(entity),entity); return entity; }
	public synchronized T findById(String id) { return data.get(id); }
	public synchronized ArrayList<T> findAll() { return new ArrayList<>(data.values()); }
	public synchronized boolean delete(String id) { return data.remove(id) != null; }
	@SafeVarargs
	public final synchronized void saveAll(T... items) {
	for (T item : items) { save(item); }
	}
	public static <N extends Number> double average(N[] values) {
	if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
	double total = 0;
	for (N n : values) { total += n.doubleValue(); }
	return total / values.length;
	}
	}

	model/Customer.java

Code	package model;
	public class Customer implements Cloneable, java.io.Serializable {
	private static final long serialVersionUID = 1L;
	public static class Address implements java.io.Serializable {
	private static final long serialVersionUID = 1L;
	private final String line, city, pincode;
	public Address(String line, String city, String pincode) {
	this.line = line; this.city = city; this.pincode = pincode;
	}
	public String getLine() { return line; }
	public String getCity() { return city; }
	public String getPincode() { return pincode; }
	}
	private Address address;
	public Customer(String name, String email, String mobile, Address address) {
	this(name,email,mobile); this.address = address;
	}
	public Address getAddress() { return address; }
	@Override
	public Customer clone() {
	try { return (Customer) super.clone(); }
	catch (CloneNotSupportedException e) { throw new AssertionError(e); }
	}
	private String name, email, mobile;
	@model.annotation.Id
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	service/Lab13.java

Code	package service;
	import model.*;
	public class Lab13 {
	public static void main(String[] args) {
	Repository<Account> accounts = new Repository<>();
	Account a = new SavingsAccount("Tej",5000,0);
	Account b = new CurrentAccount("Riya",2000,1000);
	Account c = new FixedDepositAccount("Aman",9000);
	accounts.saveAll(a,b,c);
	System.out.println(accounts.findById(a.getAccountNumber()));
	for (Account item : accounts.findAll()) { System.out.println(item); }
	Repository<Customer> customers = new Repository<>();
	Customer customer = new Customer("Tej","tej@test.com","9876543210");
	customers.save(customer);
	System.out.println(customers.findById(customer.getCustomerId()).getName());
	System.out.println(Repository.average(new Long[]{5000L,2000L,9000L}));
	}
	}

Output	

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>javac -d out exception*.java model\annotation*.java model*.java util*.java service*.java
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>java -cp out service.Lab13
AC0001 | Tej | 5000 | true
AC0001 | Tej | 5000 | true
AC0003 | Aman | 9000 | true
AC0002 | Riya | 2000 | true
Tej
5333.333333333333
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>

Practical 14

Part B	
1)	1. Create a class named BankService that uses Repository<Account>, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts. 2. Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit. 3. Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option. 4. Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct. 5. (Optional) Add an @Audit annotation and a reflective logger that records which service method was called. 6. Build the final runnable JAR and complete the README.
	Continue from Practical 13. Add BankService and the test file; replace service/MiniBank.java. Keep all other current package files. Build, run and test using the README commands below. Data is saved when Exit is selected. The JUnit tests require the external JUnit JAR.
	service/BankService.java

Code	package service;
	import model.*;
	import util.*;
	import exception.*;
	import java.nio.file.*;
	import java.io.IOException;
	import java.util.*;
	public class BankService {
	private Repository<Account> repository = new Repository<>();
	private ArrayList<String> history = new ArrayList<>();
	private Path folder;
	public BankService(Path folder) throws IOException, ClassNotFoundException {
	this.folder = folder;
	Files.createDirectories(folder.resolve("logs"));
	Path state = folder.resolve("accounts.dat");
	if (Files.exists(state)) { repository.saveAll(StatePersister.load(state)); }
	Path log = folder.resolve("logs/daily.log");
	if (Files.exists(log)) { history.addAll(Files.readAllLines(log)); }
	}
	private Account require(String number) throws AccountNotFoundException {
	Account account = repository.findById(number);
	if (account == null) { throw new AccountNotFoundException(number); }
	return account;
	}
	private void record(String type, String id, long amount) throws IOException {
	String line = type + " " + id + " " + amount;
	TransactionLog.append(folder.resolve("logs/daily.log"),line);
	history.add(line);
	}
	public synchronized Account openAccount(String type, String name, String email,
	String mobile, String pan, String ifsc, long opening) throws BankException {
	if (name == null || name.isBlank() || name.length() > 30
	|| !Validator.isValidEmail(email) || !Validator.isValidMobile(mobile)
	|| !Validator.isValidPan(pan) || !Validator.isValidIfsc(ifsc)) {
	throw new BankException("Invalid customer details");
	}
	if (opening <= 0) { throw new InvalidAmountException(); }
	Account account;
	switch (type.toUpperCase()) {
	case "SAVINGS" -> account = new SavingsAccount(name,opening,0);
	case "CURRENT" -> account = new CurrentAccount(name,opening,1000);

```
            case "FD" -> account = new FixedDepositAccount(name,opening);
            default -> throw new BankException("Invalid account type");
        }
        String[] errors = AnnotationValidator.validate(account);
        if (errors.length > 0) { throw new BankException(String.join(", ",errors)); }
        repository.save(account); return account;
    }
    public synchronized void deposit(String id, long amount) throws BankException, IOException {
        require(id).deposit(amount); record("DEPOSIT",id,amount);
    }
    public synchronized void withdraw(String id, long amount) throws BankException, IOException {
        require(id).withdraw(amount); record("WITHDRAW",id,amount);
    }
    public synchronized void transfer(String from, String to, long amount)
        throws BankException, IOException {
        Account source = require(from), target = require(to);
        source.transfer(target,amount);
        record("WITHDRAW",from,amount); record("DEPOSIT",to,amount);
    }
    public synchronized String statement(String id) throws BankException {
        StringBuilder result = new StringBuilder(StatementFormatter.buildStatement(require(id)));
        result.append("\nTransactions\n");
        for (String line : history) {
            String[] p = line.split(" ");
            if (p.length == 3 && p[1].equals(id)) { result.append(line).append("\n"); }
        }
        return result.toString();
    }
    public synchronized ArrayList<Account> listAccounts() {
        ArrayList<Account> list = repository.findAll();
        Collections.sort(list); return list;
    }
    public synchronized long balance(String id) throws BankException { return require(id).getBalance();
}
    public synchronized void save() throws IOException {
        StatePersister.save(repository.findAll().toArray(new Account[0]),folder.resolve("accounts.dat"));
    }
    public synchronized String report() throws IOException {
        return ReportGenerator.generate(folder.resolve("logs"),folder.resolve("report.txt"));
    }
    public void batchDeposit(String id, int tasks, long amount) throws Exception {
        if (tasks <= 0 || amount <= 0) { throw new InvalidAmountException(); }
        TransactionProcessor processor = new TransactionProcessor();
        processor.useAccounts(listAccounts());
        if (processor.get(id) == null) {
            processor.stop(); throw new AccountNotFoundException(id);
        }
        java.util.concurrent.ConcurrentLinkedQueue<Exception> errors =
            new java.util.concurrent.ConcurrentLinkedQueue<>();
        for (int i = 0; i < tasks; i++) {
            processor.submit(() -> {
                try { deposit(processor.get(id).getAccountNumber(),amount); }
                catch (Exception e) { errors.add(e); }
            });
        }
        processor.stop();
        if (!errors.isEmpty()) { throw errors.peek(); }
    }
}
```

service/MiniBank.java

Code

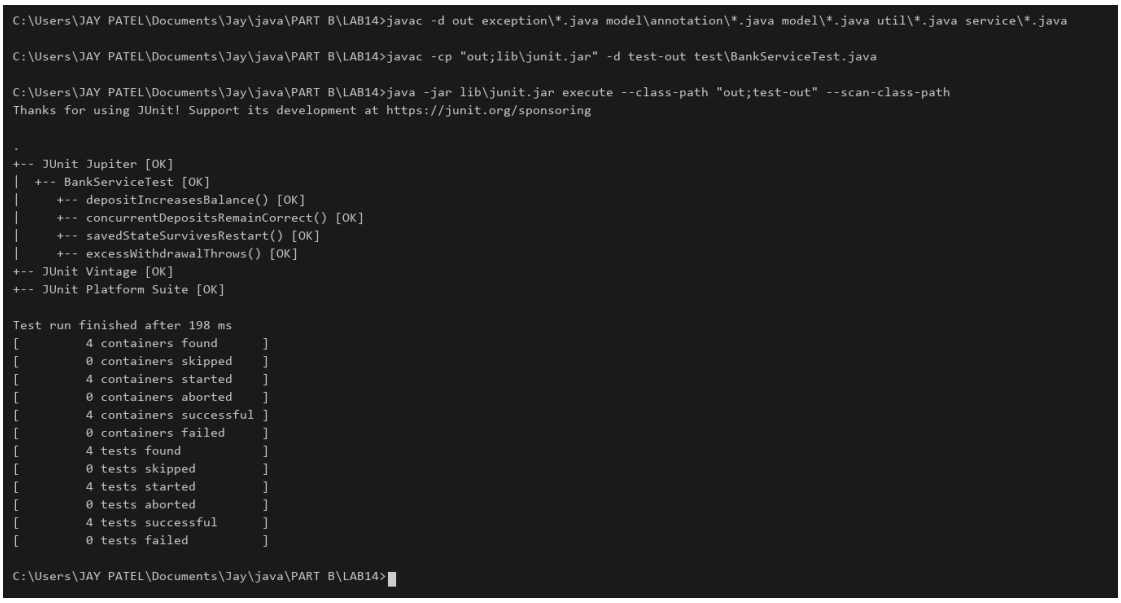
```
package service;
import model.Account;
import model.Command;
import util.*;
import java.nio.file.Path;
import java.util.Scanner;
record BankInfo(String name, String branch) {}
enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER,
```

```
            STATEMENT, LIST_ACCOUNTS, BATCH, REPORT, EXIT }
public class MiniBank {
   static String read(Scanner sc, String label) {
      System.out.print(label + ": "); return sc.nextLine().trim();
   }
   public static void main(String[] args) {
      System.out.println(new BankInfo("MiniBank","Anand"));
      try {
         BankService bank = new BankService(Path.of("data"));
         Scanner sc = new Scanner(System.in);
         boolean exit = false;
         while (!exit) {
            System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer"
               + "\n5. Statement\n6. List accounts\n7. Batch deposits\n8. Report\n9. Exit");
            try {
               int choice = Integer.parseInt(sc.nextLine());
               if (choice < 1 || choice > 9) { System.out.println("Invalid choice"); continue; }
               MenuOption option = MenuOption.values()[choice - 1];
               switch (option) {
                  case OPEN_ACCOUNT -> {
                     String type = read(sc,"Type SAVINGS/CURRENT/FD");
                     String name = read(sc,"Name");
                     String email = read(sc,"Email");
                     String mobile = read(sc,"Mobile");
                     String pan = read(sc,"PAN");
                     String ifsc = read(sc,"IFSC");
                     long opening = Long.parseLong(read(sc,"Opening balance"));
                     Account a = bank.openAccount(type,name,email,mobile,pan,ifsc,opening);
                     System.out.println("Opened: " + a);
                  }
                  case DEPOSIT, WITHDRAW -> {
                     String id = read(sc,"Account");
                     String amount = read(sc,"Amount");
                     Command c = CommandParser.parse(option.name() + " " + id + " " + amount);
                     if (option == MenuOption.DEPOSIT) {
bank.deposit(c.accountNumber(),c.amount()); }
                     else { bank.withdraw(c.accountNumber(),c.amount()); }
                     System.out.println("Balance: " + bank.balance(id));
                  }
                  case TRANSFER -> {
                     String from = read(sc,"From account");
                     String to = read(sc,"To account");
                     String amount = read(sc,"Amount");
                     Command c = CommandParser.parse("TRANSFER " + from + " " + amount);
                     bank.transfer(c.accountNumber(),to,c.amount());
                     System.out.println("Transfer completed");
                  }
                  case STATEMENT -> System.out.println(bank.statement(read(sc,"Account")));
                  case LIST_ACCOUNTS -> {
                     for (Account a : bank.listAccounts()) { System.out.println(a); }
                  }
                  case BATCH -> {
                     String id = read(sc,"Account");
                     int tasks = Integer.parseInt(read(sc,"Number of deposits"));
                     long amount = Long.parseLong(read(sc,"Amount per deposit"));
                     bank.batchDeposit(id,tasks,amount);
                     System.out.println("Balance: " + bank.balance(id));
                  }
                  case REPORT -> System.out.println(bank.report());
                  case EXIT -> {
                     bank.save(); exit = true; System.out.println("Saved. Goodbye");
                  }
               }
            } catch (Exception e) { System.out.println("Error: " + e.getMessage()); }
         }
      } catch (Exception e) { System.out.println("Startup failed: " + e.getMessage()); }
   }
}
```

	test/BankServiceTest.java

Code	package test;
	import service.BankService;
	import model.Account;
	import exception.*;
	import java.nio.file.Path;
	import org.junit.jupiter.api.Test;
	import org.junit.jupiter.api.io.TempDir;
	import static org.junit.jupiter.api.Assertions.*;
	public class BankServiceTest {
	@TempDir Path folder;
	Account open(BankService bank) throws BankException {
	return bank.openAccount("SAVINGS","Tej","tej@test.com","9876543210",
	"ABCDE1234F","SBIN0001234",1000);
	}
	@Test void depositIncreasesBalance() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.deposit(a.getAccountNumber(),500);
	assertEquals(1500,bank.balance(a.getAccountNumber()));
	}
	@Test void excessWithdrawalThrows() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank);
	assertThrows(InsufficientFundsException.class,
	() -> bank.withdraw(a.getAccountNumber(),5000));
	assertEquals(1000,bank.balance(a.getAccountNumber()));
	}
	@Test void concurrentDepositsRemainCorrect() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.batchDeposit(a.getAccountNumber(),100,10);
	assertEquals(2000,bank.balance(a.getAccountNumber()));
	}
	@Test void savedStateSurvivesRestart() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.deposit(a.getAccountNumber(),500); bank.save();
	BankService loaded = new BankService(folder);
	assertEquals(1500,loaded.balance(a.getAccountNumber()));
	Account next = open(loaded);
	assertNotEquals(a.getAccountNumber(),next.getAccountNumber());
	}
	}

	README.txt
Text	MiniBank requires JDK 17 or newer. Keep the package folders model, model/annotation, util, service and exception together. Retain the files from Practicals 6 to 13 and apply the additions and replacements from Practical 14. Earlier LabNN demo classes may remain; run service.MiniBank for the final application. Build: javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java jar cfm minibank.jar manifest.mf -C out . Run: java -jar minibank.jar Choose 1 to open an account. Enter SAVINGS, CURRENT or FD, a nonblank name of at most 30 characters, a valid email, a 10-digit mobile starting 6 to 9, PAN ABCDE1234F, IFSC SBIN0001234, and a positive opening balance. Savings uses a minimum balance of 0; current allows an overdraft of 1000; fixed deposit withdrawals are locked. Options 2 to 4 perform transactions. Options 5 and 6 show statements and accounts. Option 7 submits a batch of deposits to a four-thread pool. Option 8 writes a report. Option 9 saves accounts and exits. The next launch loads data/accounts.dat. Logs are appended to data/logs/daily.log. Transfer logs contain one withdrawal and one deposit, so the net change is zero. Opening balances are initial capital and are excluded from transaction totals. Report totals cover all retained log entries. JUnit tests:

	Download JUnit Platform Console Standalone 1.10.2 and save it as lib/junit.jar. Test sources stay in test/ and are not included in the application JAR. javac -cp "out;lib/junit.jar" -d test-out test/BankServiceTest.java java -jar lib/junit.jar execute --class-path "out;test-out" --scan-class-path Use a colon instead of a semicolon in classpaths on Linux/macOS. Git: Ignore out/, test-out/, data/, lib/ and *.jar. Commit the source, manifest and README. Tag the final version lab-14. Earlier stages use tags lab-01 to lab-13.
Output	

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>javac -cp "out;lib\junit.jar" -d test-out test\BankServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>java -jar lib\junit.jar execute --class-path "out;test-out" --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- BankServiceTest [OK]
|     +-- depositIncreasesBalance() [OK]
|     +-- concurrentDepositsRemainCorrect() [OK]
|     +-- savedStateSurvivesRestart() [OK]
|     +-- excessWithdrawalThrows() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 198 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         4 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>
```